理解大模型开发（二）-实战篇

- 来源: https://class.imooc.com/lesson/5267#mid=104904
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第4周 理解大模型开发
- 模块: 理解大模型开发
- 课节: 图文 1-3 理解大模型开发（二）-实战篇

1-3 理解大模型开发（二）-实战篇

理解大模型开发（二）-实战篇

> [!TIP] 阅读指南 上一章我们建立了认知框架：大模型擅长理解和生成，但不擅长确定性计算。现在的问题是，如何在真实系统中落地？ 本章将通过一个完整的智能客服案例，展示AI和传统技术如何协作。然后讨论两种LLM接入方式的选择策略，最后探讨从确定性编程到概率性编程的思维转变。 这是从理论到实践的关键一步。

3.1 混合架构：智能客服案例

用一个完整的例子，展示AI和传统技术如何协作。

假设要做一个电商网站的智能客服。用户可能会问：

- “我的订单什么时候发货？”
- “这件衣服有XL码吗？”
- “可以退货吗？”

纯靠AI来回答？成本高，而且容易出错（AI可能"编造"一个订单状态）。

更好的方案是:让AI和传统技术接力完成任务。 下述流程里会有少量的代码，你无需研究，这里只是希望同学们提前感受下调用LLM的代码风格。

四棒接力赛

- 第一棒:AI理解意图

用户输入:“我的订单什么时候发货?”

调用大模型API,识别意图:

```
response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "你是意图分类助手，判断用户意图，只返回：查询订单、查询商品、退换货、支付问题、其他"},
        {"role": "user", "content": "我的订单什么时候发货?"}
    ]
)
# 返回："查询订单"
```

大模型把自然语言"翻译"成了明确的意图标签。

- 第二棒:传统技术提取信息

识别出是"查询订单"后，需要找到订单号。

假设用户的完整输入是：“我的订单号：2024011500123”。

这不需要LLM，用正则表达式就行:

```
import re

user_message = "请帮我查一下订单2024011500123的物流状态"
order_number = re.search(r'\d{10,}', user_message)
# 提取出：2024011500123
```

快速、准确、成本为零。

- 第三棒:传统技术查询数据

拿到订单号，去数据库查询:

```
order = db.query("SELECT * FROM orders WHERE id = ?", order_number)

1
```

返回:

```
{
  "status": "已发货",
  "tracking": "SF1234567890",
  "eta": "1月15日"
}
```

数据库查询是确定的、精确的、可靠的。订单状态来自真实数据，不会出错。

- 第四棒: LLM生成回复

最后，把结构化数据转化为友好的自然语言:

```
response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "system", "content": "你是客服助手，根据订单信息生成友好回复"},
        {"role": "user", "content": f"订单状态:{order['status']}, 物流单号:
{order['tracking']}, 预计送达:{order['eta']}"}
    ]
)

# 返回："您的订单已发货，物流单号是SF1234567890，预计1月15日送达，请注意查收！"
```

我们可以看到在这个过程中，LLM只是一个辅助角色，协助主系统完成它不擅长的事情（自然语言分析）：

- 把模糊的用户语言提炼成确定的意图：“查询订单”。
- 把数据库里的冷冰冰的数据翻译成人类更喜欢的自然语言。

但主要的处理逻辑依然是传统系统在主导。

3.2 两种LLM接入方式

确定了哪些功能需要LLM，下一个问题是：怎么接入LLM?

怎么接入LLM?

目前主流的方式有两种:调用LLM API,或者自己部署模型。

方式一:调用API

各大大模型厂商(OpenAI、Google、DeepSeek、阿里等)都提供了API(Application Programming Interface，应用程序编程接口)服务。你只需要几行代码就可以完成调用:

```
from openai import OpenAI

client = OpenAI(api_key="你的密钥")

response = client.chat.completions.create(
    model="gpt-5",
    messages=[
        {"role": "user", "content": "帮我写一段商品介绍"}
    ]
)

print(response.choices[0].message.content)
```

就这么简单。不需要GPU服务器，不需要下载数十GB的模型文件，不需要研究推理优化。

这种方式好处包括：零基础设施投入，模型自动升级（可以看到，上述代码里我们指定了model="gpt-5"），按需付费。但也有缺点：调用量大时成本高，数据要发送到云端存在隐私风险，无法深度定制。

方式二:自建模型

下载开源模型(如Meta的LLaMA、阿里的Qwen(通义千问)、深度求索的DeepSeek),部署到自己的服务器上。

优势包括：数据不出本地，隐私可控；长期成本低，只有电费和硬件折旧；可以微调模型，适配特定领域。

劣势也很明显：前期投入太大，一台好的GPU服务器可能要几十万；技术门槛高，需要掌握模型部署、量化、推理优化；需要专人维护；开源模型能力通常弱于顶级商业模型。

如何选择?

> 个人开发者 / 小团队 / 创业公司 → 先用API,快速验证产品 数据敏感行业(医疗、金融、政务) 调用量巨大(每天百万次以上) 需要深度定制 → 考虑自建 不确定? → 从API开始，等业务稳定后再决定

很多公司采用混合方案:

混合方案

- 日常的、标准的任务用自建模型(成本低)
- 复杂的、偶尔的任务调用顶级API(能力强)

就像平时自己做饭，偶尔请客去餐厅。

3.3 思维方式的转变

使用LLM开发应用，最大的挑战不是技术，而是思维方式的转变。

思维方式的转变

- 传统编程是确定性的世界

在传统编程中，一切都是确定的:

```
def calculate_total(price, quantity):
    return price * quantity

1
2
```

输入相同，输出必然相同。10元一件，买3件，永远是30元。不会今天是30,明天变成29.8。

你能精确控制每一个逻辑分支，能预测每一行代码的行为。调试的时候，找到出错的那一行，改掉，问题就解决了。

这种确定性，给了我们掌控感。

- LLM是概率性的世界

但大模型不是这样。

你让LLM写一段商品介绍:

```
response = ai.generate("给这款防晒霜写个介绍")

1
```

"给这款防晒霜写个介绍"

第一次调用，它可能写:“这款防晒霜清爽不油腻，适合夏日使用。”

第二次调用，同样的输入，它可能写:“轻盈质地，SPF50+高倍防护，让你的肌肤远离紫外线伤害。”

内容不一样，但都算"对"。这就是AI的概率性——它不是在"计算"答案，而是在"生成"内容。每次生成，都从概率分布中采样，所以结果可能不同。

观点需要从三个方面转变。

- 接受不完美

LLM偶尔会"胡说八道"。需要加上验证机制：

```
response = ai.generate("...")

# 检查生成的内容是否合理
if not is_valid(response):
    # 重试,或使用备用方案
    response = fallback_response
```

- 学会"引导"而非"控制"

不能说的："给我生成长度恰好50个字的介绍。"只能说的：“请写一段大约50字左右的介绍。”

LLM可能生成48个字，可能53个字，甚至偶尔70个字。没法精确控制，只能通过优化提示词来引导它靠近期望。

- 持续监控和迭代

AI的表现会波动。用户的需求也在变化。需要持续观察哪些回复用户不满意，哪些场景AI处理得不好，提示词是否需要调整，是否需要换用更强的模型。

这和传统的"写完代码就稳定运行"完全不同。LLM应用更像一个需要持续照料的花园，而不是一台一次装配好就能永久运转的机器。

但这也意味着，新的可能性。

正是因为AI的"不确定性"，它才有了"创造性"。它能写出你没想到的文案，能发现你没注意到的模式，能给出你没考虑过的方案。

从确定性到概率性，不是退步，而是进化。我们学会了与一种新的智能共处，学会了在"引导"中发挥它的潜力，也在这个过程中，重新思考什么是"控制"，什么是"协作"。

3.4 下节预告

理解了LLM的定位、架构和思维方式后，你可能已经跃跃欲试：

那么，如何真正跟大模型"对上话"？

你需要什么？API密钥？代码库？服务器？第一次调用会遇到哪些坑？怎么验证是否调用成功？

下一节《第一次API调用》，我们将从零开始，一步步带你完成第一次与LLM的真实对话。

3.5 ■ 学点英语

中文	English	音标	说明
混合架构	Hybrid Architecture	/ˈhaɪbrɪd ˈɑːrkɪtektʃər/	AI和传统技术接力协作的系统架构模式
四棒接力赛	Four-Baton Relay	/fɔːr ˈbætn ˈriːleɪ/	AI理解意图→传统提取信息→传统查询数据→LLM生成回复的协作流程
调用API	API Calling	/ˈeɪpiːaɪ ˈkɔːlɪŋ/	通过API调用云端大模型，零基础设施投入的接入方式
自建模型	Self-Hosted Model	/self ˈhoʊstɪd ˈmɑːdl/	下载开源模型部署到自有服务器，数据不出本地
概率性编程	Probabilistic Programming	/ˌprɑːbəbɪˈlɪstɪk ˈproʊgræmɪŋ/	接受AI输出的不确定性，通过引导而非控制的开发范式